SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Oral Defense week 6

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY –2026

Contents

Oral Defense week 6 2

1. Explain the implementation of your Candlestick chart. What information do the "wicks" and the "body" of the candle provide compared to a standard line chart? 3

2.In your Boxplot visualization, you use a "moving window." How does visualizing the distribution of prices over n days help in identifying market volatility? 4

3. Why is it useful to save these visualizations as images if you were to use a CNN-based approach for prediction in the future? 5

Oral Defense week 6

1. Explain the implementation of your Candlestick chart. What information do the "wicks" and the "body" of the candle provide compared to a standard line chart?

```python
def aggregate_ohlcv_n_days(df_ohlcv: pd.DataFrame, n_days_per_candle: int = 1) -> pd.DataFrame:
    """
    Aggregate OHLCV data so that each "candle" represents n trading days (n >= 1).

    Why we need this:
    - Task C.3 requires an option for 1 candlestick to express data of n trading days.
    - Trading days are rows in the dataset (not calendar days), so we aggregate consecutive rows.

    Aggregation rules for n-day candles:
    - Open   = first Open in the group
    - High   = max High in the group
    - Low    = min Low in the group
    - Close  = last Close in the group
    - Volume = sum Volume in the group (if present)

    Args:
        df_ohlcv: A dataframe containing at least Open/High/Low/Close and optionally Volume.
                  Index should be DatetimeIndex.
        n_days_per_candle: number of trading days per candle. If 1, no aggregation is applied.

    Returns:
        Aggregated dataframe with the same OHLC(V) column names and a DatetimeIndex.
    """
    if n_days_per_candle < 1:
        raise ValueError("n_days_per_candle must be >= 1")

    df = _flatten_yfinance_columns(df_ohlcv).copy()
```

How I implemented it:

- I plot an OHLCV dataframe using a candlestick chart.
- I added an option n_days_per_candle (n ≥ 1) by aggregating OHLCV into n-day blocks:
    - Open = first open in the block
    - High = max high
    - Low = min low
    - Close = last close
    - Volume = sum volume
    - This creates candles that represent 1 day (n=1) or multi-day summaries (e.g., n=5).

```python
    agg_dict = {
        "Open": "first",
        "High": "max",
        "Low": "min",
        "Close": "last",
    }
    if "Volume" in df.columns:
        agg_dict["Volume"] = "sum"

    grouped = df.groupby(group_id).agg(agg_dict)

    # Use the first date in each group as the candle timestamp
    first_dates = df.groupby(group_id).apply(lambda x: x.index[0])
    grouped.index = pd.to_datetime(first_dates.values)

    return grouped
```

Wicks and body meaning:

- The body shows the range between Open and Close.

    - If Close > Open → bullish (price increased during that period)
    - If Close < Open → bearish (price decreased)
- The wicks show the extremes:
    - Upper wick = distance from max(Open, Close) to High
    - Lower wick = distance from min(Open, Close) to Low. These indicate how far price moved beyond open/close during the period.

Why this is richer than a line chart:

- A standard line chart often plots only Close, which hides intraday extremes.
- Candlesticks reveal:
    - volatility (long wicks)
    - price rejection (wick spikes that reverse)
    - momentum (large bodies)
    - So candlesticks preserve more market structure than “Close-only” lines.

2.In your Boxplot visualization, you use a "moving window." How does visualizing the distribution of prices over n days help in identifying market volatility?

How I implemented it:

- I create a series of boxplots from a moving window:
    - window_size = n days (e.g., 20)
    - step controls how far the window shifts each time (e.g., 5). Each boxplot is the distribution of prices in that window.

```python
    # Build rolling windows
    for start in range(0, len(series) - window_size + 1, step):
        end = start + window_size
        windows.append(series.iloc[start:end].values)
        labels.append(str(series.index[start].date()))

    # Keep only the last max_windows windows (most recent period)
    if max_windows is not None and len(windows) > max_windows:
        windows = windows[-max_windows:]
        labels = labels[-max_windows:]

    plt.figure(figsize=(12, 5))
    plt.boxplot(windows, showfliers=False)

    title = f"{company} Moving-Window Boxplot ({price_column}) | window={window_size}, step={step}"
    plt.title(title)
    plt.xlabel("Window (start date labels sampled)")
    plt.ylabel("Price")
```

How it helps detect volatility:

- Boxplots summarize a window using:
    - median (middle line),
    - interquartile range (IQR: Q3–Q1, “box height”),
    - whiskers (spread outside IQR),
    - outliers (if shown).
- Higher volatility windows show:
    - taller boxes (bigger IQR),
    - longer whiskers (wider range),
    - more frequent outliers (sharp moves).
- As you move across time windows, you can visually see when volatility expands or compresses—much easier than reading raw numbers.

3. Why is it useful to save these visualizations as images if you were to use a CNN-based approach for prediction in the future?

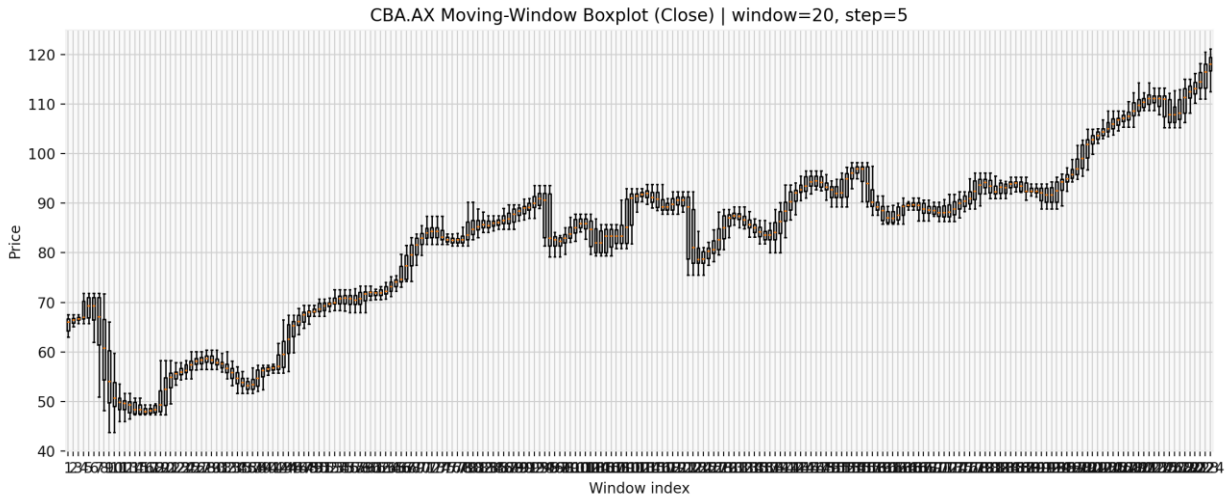
Saving visualizations as images is useful because:

- CNNs are designed for image inputs, not raw sequences.
- If I convert time-series data into images (candlestick charts, boxplots), I can treat it like a pattern recognition task:
    - CNN can learn shapes like long wicks, large bodies, trend formations, and volatility regimes.
- Saving images also supports:
    - building a labeled dataset (e.g., “price up/down next day”),
    - consistent preprocessing (same chart size/style),
    - offline augmentation (crop, blur, resize) for model robustness.

In short: saving charts as PNGs creates a bridge from traditional time-series data to vision-based learning methods.

CBA.AX Candlestick (n=5 trading day(s)/candle)

CBA.AX Candlestick (n=1 trading day(s)/candle)


CBA.AX Moving-Window Boxplot (Close) | window=20, step=5
Price
Window index
120
110
100
90
80
70
60
50
40